

Session 09 Homework — Finish the Trim Fix

CORE, STRETCH, and CHALLENGE are all required — every one of them must be completed. The three names describe increasing difficulty only, nothing about which parts you submit.

Context

This extends B4F Board's `api/` exactly where class left off: `CreateProjectDto` / `CreateWorkspaceDto` trim and validate `name` / `key` / `slug` with `@Transform()` + `class-validator`, and `UpdateProjectDto` / `UpdateWorkspaceDto` both inherit those same rules via `extends PartialType(...)` — built in class and in the teamwork.

Goal

Apply the identical `@Transform(({ value }) => value.trim())` pattern to the one field it hasn't reached yet — `description` — and then explain, in writing, why the two identifying fields (`key`, `slug`) needed it most of all.

What to build

A trimmed `description` field on `CreateProjectDto` (CORE), the identical fix on `CreateWorkspaceDto` (STRETCH), and a short written comment on `key` / `slug` in both files explaining why trimming an identifying field matters more than trimming a descriptive one (CHALLENGE).

CORE (required)

Add `@Transform(({ value }) => value.trim())` to `description?` in `CreateProjectDto` — the same decorator, same position (between `@IsString()` and the end of the field), that `name` / `key` already have. Do **not** add `@IsNotEmpty()` — an empty description is still allowed; only the whitespace itself needs trimming.

Because `UpdateProjectDto` extends `PartialType(CreateProjectDto)`, you do not need to touch `update-project.dto.ts` at all — the fix reaches `PATCH` automatically.

STRETCH (required)

Do the identical thing for Workspaces: `@Transform(({ value }) => value.trim())` on `description?` in `CreateWorkspaceDto`. Same reasoning — `UpdateWorkspaceDto` inherits it for free.

CHALLENGE (required)

`key` (on `CreateProjectDto`) and `slug` (on `CreateWorkspaceDto`) already have the trim fix from class. For each one, add a short code comment directly above the field explaining, in your own words, why trimming THIS field matters more than trimming `description`: both `ProjectsService` and `WorkspacesService` check `key` / `slug` for uniqueness, so an untrimmed value could otherwise coexist with its trimmed twin as if they were different values.

Test it: `PATCH` a project's `key` to `" WEB"` (a leading space) where `"WEB"` already exists on another project, and confirm it is correctly treated as the SAME key (a `409 Conflict`), not a new one.

Rules

Allowed: everything taught in Session 09 — `@Transform()`, exactly as used in class and the teamwork, applied to one more field per DTO; a code comment (no new decorators, no new packages).

Not yet: no new decorators beyond `@Transform()`, no `PickType / OmitType` (FLEX-only, not required), no guards, no database, no custom pipes.

Test it

BASH

```
curl -X POST http://localhost:3000/projects -H "Content-Type: application/json" -d '{"name": "Docs Site", "key": "DOCS", "description": " Public documentation  "'}
curl -X PATCH http://localhost:3000/projects/1 -H "Content-Type: application/json" -d '{"key": " WEB"}'
```

Done when

- A `POST / PATCH` with a padded `description` stores and returns it trimmed, on both Projects and Workspaces.
- A `PATCH` with a leading/trailing-space `key / slug` that collides with an existing one (once trimmed) correctly returns `409`, not `200`.
- `npm run start:dev` starts with no errors and no warnings in the terminal.
- You can explain, out loud, why `key / slug` needed the comment and `description` didn't.

Working time

Expect roughly 20-30 minutes total across all three tiers.